

Tank Shield - Wiring Guide e configurazione

Firmware: [Embedded_Project_Tank_2025_2026](#)
Tank: Arduino Uno R4 WiFi + Emakefun PS2X & Motor Drive Board
Controller: ESP32 Dev Module
Aggiornamento: 19 luglio 2026

1. Mappa generale

Funzione	Collegamento	Note
Cingolo sinistro	Shield M3	Motore DC a 2 fili
Cingolo destro	Shield M1	Motore DC a 2 fili
Porta non usata	Shield M2	Lasciata libera perche' sull'hardware provato girava da sola
Torretta orizzontale	D2-D5 verso driver IN1-IN4	Stepper 5 fili con driver esterno
Servo elevazione A	Shield S5	Angolo logico diretto 0-47 gradi
Servo elevazione B	Shield S6	Movimento specchiato: 90 - angolo
Relay cannone	D7 verso pin S del relay	Morsetti di potenza COM e NO
Comunicazione	WiFi AP + UDP	Tank_AP , password 12345678 , porta 4210

I motori e i servo sono comandati dalla shield. La torretta orizzontale e il relay usano i pin digitali Arduino riportati sui connettori della shield.

2. Cingoli DC

Ogni motore giallo ha due fili e deve usare entrambi i morsetti della stessa porta motore. Non collegare un filo del motore a GND.

Cingolo	Porta	Collegamento
Sinistro	M3	I due fili del motore nei due morsetti di M3
Destro	M1	I due fili del motore nei due morsetti di M1

Se un cingolo gira al contrario, scambia i suoi due fili oppure modifica in [tank/src/trackController.h](#) :

```
#define LEFT_TRACK_INVERTED false
#define RIGHT_TRACK_INVERTED false
```

Metti [true](#) soltanto sul lato che deve cambiare direzione.

3. Torretta orizzontale

Lo stepper a 5 fili non e' collegato direttamente alla shield: il suo driver esterno alimenta le bobine. Arduino manda soltanto la sequenza digitale sui quattro ingressi.

Arduino/shield	Driver torretta
D2	A / IN1
D3	B / IN2
D4	C / IN3
D5	D / IN4
GND	GND driver

Alimenta il driver con la tensione richiesta dal suo modulo. Il suo GND deve essere collegato al GND comune di Arduino, shield, ESP32/alimentazioni collegate via cavo e buck converter.

Il firmware usa la sequenza full-step a una fase:

```
A -> B -> C -> D -> A      rotazione in un verso
D -> C -> B -> A -> D      rotazione nel verso opposto
```

La velocita dipende da `TURRET_STEP_INTERVAL_MS` in `tank/src/servoTorreta.h`: un valore piu' grande rallenta la torretta. Il valore attuale e' `2 ms`.

4. Servo elevazione

Il connettore servo della shield ha tre file: `G` = massa, `V` = alimentazione, `S` = segnale. Controlla sempre le lettere stampate sulla scheda prima di inserire il connettore.

Servo	Porta	Comando attuale
Servo A	S5	da 0 a 47 gradi
Servo B	S6	da 90 a 43 gradi, cioe' <code>90 - angolo A</code>

Il valore mostrato sulla seriale del tank e' l'angolo logico del servo A. Il servo B riceve automaticamente il comando specchiato.

Configurazione in `tank/src/servoTorreta.cpp`:

```
#define ELEVATION_MIN_ANGLE 0
#define ELEVATION_MAX_ANGLE 47
#define ELEVATION_MIRROR_BASE 90
```

5. Relay e cannone

Lato controllo

Relay	Arduino/shield
+	5V
-	GND
S	D7

Lato potenza

Usa COM e NO, non NC:

```
positivo alimentazione cannone -> COM relay
NO relay                        -> positivo del cannone
negativo alimentazione        -> negativo del cannone
```

Guardando il modulo come nella foto del progetto, il morsetto superiore verso il cannone e' NO. Verifica comunque la scritta NO/COM/NC sul PCB prima di alimentare.

Il relay riceve un impulso di 200 ms. Tra due colpi devono passare almeno 12 secondi, anche se il pulsante viene premuto piu' volte.

6. Controller ESP32

Collegamenti fisici

Componente	Uscita fisica	Pin ESP32
Joy1 guida	asse X	GPIO 34
Joy1 guida	asse Y	GPIO 32
Joy2 torretta	asse X	GPIO 35
Joy2 torretta	asse Y	GPIO 33
Pulsante zero	segnale	GPIO 25, pulsante verso GND
Pulsante sparo	segnale	GPIO 26, pulsante verso GND

I pulsanti usano INPUT_PULLUP: rilasciato = HIGH, premuto = LOW.

Scambio X/Y nel software

I joystick sono montati ruotati. Il firmware scambia X e Y su entrambi:

```
#define DRIVE_SWAP_X_Y true
#define TURRET_SWAP_X_Y true
```

Dopo lo scambio, i comandi logici inviati via UDP sono:

Comando logico	Pin fisico letto	Funzione
driveX	GPIO 32	sterzata differenziale
driveY	GPIO 34	avanti/indietro
turretX	GPIO 33	torretta orizzontale
elevationY	GPIO 35	elevazione servo

Per invertire soltanto il verso di un asse usa le quattro costanti `DRIVE_X_INVERTED`, `DRIVE_Y_INVERTED`, `TURRET_X_INVERTED` ed `ELEVATION_Y_INVERTED` in `controller-esp32/src/joystickReader.cpp`. L'orientamento degli assi Joy1 viene corretto solo qui; sul tank restano soltanto le inversioni dei due motori fisici (`LEFT/RIGHT_TRACK_INVERTED`).

7. Calibrazione joystick

All'avvio dell'ESP32:

1. Lascia entrambi i joystick fermi al centro.
2. Il firmware aspetta `600 ms`.
3. Legge ogni asse `80` volte, a distanza di `2 ms`, e calcola il centro reale. La calibrazione completa richiede quindi circa `1,24 s` piu' il piccolo overhead di elaborazione.
4. Accetta la calibrazione solo se ogni asse e' vicino a `512` (massimo scostamento `70`) e non oscilla oltre `40` punti durante il campionamento.
5. Rimappa il centro misurato a `512`, mantenendo disponibili gli estremi `0-1023`. Se la calibrazione non e' valida, invia solo il comando sicuro e richiede un riavvio con joystick fermi.
6. Anche con calibrazione valida, prima di armare i comandi richiede joystick neutrali e pulsanti rilasciati per `400 ms`.

Le deadzone non sono tutte uguali: ogni filtro ha uno scopo diverso.

Filtro	Valore e zona ferma
Joy1 guida sul controller	<code>DRIVE_INPUT_DEADZONE = 20</code> : meno di <code>20</code> punti dal centro calibrato
Joy2 torretta/elevazione sul controller	<code>TURRET_INPUT_DEADZONE = 80</code> : meno di <code>80</code> punti dal centro calibrato
Cingoli sul tank	<code>TRACK_COMMAND_DEADZONE = 100</code> : meno di <code>100</code> punti dal centro UDP, prima e dopo il mixing
Torretta e servo sul tank	<code>TURRET_JOYSTICK_DEADZONE = 200</code> : meno di <code>200</code> punti dal centro UDP

La deadzone Joy1 piccola evita di sommare una zona morta eccessiva a quella di protezione del tank. Non toccare i joystick durante la calibrazione iniziale e durante l'attesa di armamento.

8. Guida differenziale e velocita

Joy1 Y controlla avanti/indietro in modo lineare e puo' raggiungere la velocita massima. Joy1 X controlla soltanto la sterzata. Non esistono piu' curve, fasce o soglie intermedie di sterzo.

```
forward = decode(driveY)
turn     = decode(driveX)
sinistro = constrain(forward + turn, -512, +512)
destro   = constrain(forward - turn, -512, +512)
```

`decode()` limita l'ingresso alla scala `0-1023`, sottrae il centro `512` e applica la deadzone del tank. A joystick Y in avanti entrambi i cingoli ricevono lo stesso comando; con Joy1 X un lato accelera e l'altro rallenta. Con il solo Joy1 X, i due cingoli girano in versi opposti e il tank ruota sul posto. Il `constrain()` evita che la somma superi il comando massimo consentito.

Configurazioni principali in `tank/src/trackController.h`:

```
#define TRACK_COMMAND_DEADZONE 100
#define LEFT_TRACK_MIN_PWM 900
#define RIGHT_TRACK_MIN_PWM 900
```

Quando viene richiesta un'inversione di marcia, ogni cingolo viene prima frenato a PWM zero e riattivato dopo `30 ms` (`TRACK_REVERSE_DEAD_TIME_MS` in `tank/src/trackController.cpp`). La pausa vale anche se il joystick passa brevemente dal centro prima dell'altro verso.

9. Protocollo UDP e frequenza

Il controller invia circa `100 pacchetti al secondo`, uno ogni `10 ms`:

```
V1;driveX;driveY;turretX;elevationY;zero;fire
```

Esempio:

```
V1;512;900;512;512;0;0
```

Il tank conserva l'ultimo comando valido. Se non riceve pacchetti per `500 ms`, centra tutti gli assi, disattiva i pulsanti e ferma i movimenti.

All'avvio il tank resta disarmato. Se `WiFi.beginAP()` o l'apertura del socket UDP falliscono, non entra in `while(true)`: resta nello stato sicuro e ritenta ogni `3 secondi`. Quando AP e UDP sono pronti, e dopo ogni boot, timeout o guasto rete, il tank richiede `3 pacchetti consecutivi` con tutti gli assi entro `+/-20` dal centro e pulsanti rilasciati prima di riarmare gli attuatori.

Se l'ESP32 perde `Tank_AP`, non blocca il programma: interrompe l'invio, tenta la riconnessione ogni `3 secondi` e lascia che il tank entri nel timeout di sicurezza. Quando il WiFi torna disponibile, aggiorna l'IP dal gateway, riapre UDP sulla porta `4210`, richiede di nuovo la neutralità del controller e riprende automaticamente l'invio solo dopo il riarmo.

10. Alimentazione

Secondo il manuale Emakefun V1.4:

- la shield può essere alimentata dal jack DC di Arduino con `6-12 V`;
- la batteria da `7,4 V` sul jack Arduino rientra quindi nell'intervallo previsto;
- con jumper servo su `5V`, i servo usano il regolatore a 5 V della shield;
- con jumper su `EX`, i servo usano l'ingresso di alimentazione esterno;
- il manuale indica fino a `3 A` per il ramo 5 V della shield, ma il consumo reale dei due servo dipende dal carico e dai picchi di stallo.

Configurazione consigliata quando i servo fanno scatti o resettano Arduino:

```
batteria 7,4 V -> jack DC Arduino
batteria 7,4 V -> ingresso buck converter
uscita buck regolata -> alimentazione servo esterna della shield
jumper servo -> EX
GND buck -> GND Arduino/shield
```

Regola e misura l'uscita del buck prima di collegare i servo. Usa la tensione ammessa dai servo, normalmente 5-6 V. Non collegare due alimentazioni diverse allo stesso ramo positivo e non alimentare motori o servo dalla sola USB.

11. Dove modificare il comportamento

Modifica	File	Costante/funzione
Scambiare X/Y Joy1	<code>controller-esp32/src/joystickReader.cpp</code>	<code>DRIVE_SWAP_X_Y</code>
Scambiare X/Y Joy2	<code>controller-esp32/src/joystickReader.cpp</code>	<code>TURRET_SWAP_X_Y</code>
Invertire un asse	<code>controller-esp32/src/joystickReader.cpp</code>	costanti <code>*_INVERTED</code>
Deadzone Joy1 guida	<code>controller-esp32/src/joystickReader.cpp</code>	<code>DRIVE_INPUT_DEADZONE</code>
Deadzone Joy2	<code>controller-esp32/src/joystickReader.cpp</code>	<code>TURRET_INPUT_DEADZONE</code>
Deadzone cingoli	<code>tank/src/trackController.h</code>	<code>TRACK_COMMAND_DEADZONE</code>
Velocità singolo cingolo	<code>tank/src/trackController.h</code>	<code>LEFT/RIGHT_TRACK_MIN/MAX_PWM</code> , <code>LEFT/RIGHT_TRACK_PWM_PERCENT</code>
Pausa prima dell'inversione	<code>tank/src/trackController.cpp</code>	<code>TRACK_REVERSE_DEAD_TIME_MS</code>
Direzione cingolo	<code>tank/src/trackController.h</code>	<code>LEFT/RIGHT_TRACK_INVERTED</code>
Velocità torretta	<code>tank/src/servoTorretta.h</code>	<code>TURRET_STEP_INTERVAL_MS</code>
Limiti elevazione	<code>tank/src/servoTorretta.cpp</code>	<code>ELEVATION_MIN/MAX_ANGLE</code>
Specchio secondo servo	<code>tank/src/servoTorretta.cpp</code>	<code>ELEVATION_MIRROR_BASE</code>
Durata impulso relay	<code>tank/src/main.cpp</code>	<code>FIRE_RELAY_PULSE_MS</code>
Pausa fra colpi	<code>tank/src/main.cpp</code>	<code>FIRE_RELAY_COOLDOWN_MS</code>
Retry AP/UDP del tank	<code>tank/src/udpReceiver.cpp</code>	<code>NETWORK_RETRY_INTERVAL_MS</code>
Riarmo UDP del tank	<code>tank/src/udpReceiver.cpp</code>	<code>REARM_NEUTRAL_PACKET_COUNT</code> , <code>REARM_NEUTRAL_TOLERANCE</code>

12. Schema rapido

ESP32 controller

```
GPI034 <- Joy1 X fisico -> driveY dopo lo scambio
GPI032 <- Joy1 Y fisico -> driveX dopo lo scambio
GPI035 <- Joy2 X fisico -> elevationY dopo lo scambio
GPI033 <- Joy2 Y fisico -> turretX dopo lo scambio
GPI025 <- pulsante zero verso GND
GPI026 <- pulsante sparo verso GND
      |
      | WiFi UDP, ogni 10 ms
      v
```

Arduino Uno R4 WiFi + shield

```
M3      -> motore DC cingolo sinistro, entrambi i fili
M1      -> motore DC cingolo destro, entrambi i fili
D2-D5   -> IN1-IN4 driver stepper torretta
S5      -> servo elevazione A
S6      -> servo elevazione B
D7      -> S relay cannone
Relay   -> COM + NO, NO verso il cannone
```

13. Verifica prima dell'accensione

- Nessun filo motore collegato direttamente a GND.
- Driver torretta e Arduino hanno GND comune.
- Connettori servo orientati secondo [G/V/S](#) stampati sulla shield.
- Buck misurato con multimetro prima dei servo.
- Jumper servo nella posizione coerente: [5V](#) oppure [EX](#).
- Relay collegato a [COM](#) e [NO](#), non a [NC](#).
- Joystick lasciati al centro durante l'avvio dell'ESP32.